Lecture 16

- **Stored Program –** instructions and data come from a memory component.
- **Sequential instruction processing –** instructions drawn from memory, one after the other using a program counter (PC) to keep track of where we are in the list of instructions.
- **Instruction Set Architecture –** interface between the highest-level hardware and the lowest level software.
  - Contract between programmer and CPU designer to hide details from each other so they can work independently.

Lecture 17

- **Visible state -** registers that are part of the ISA so both hardware and software can see it (tends to be R0 - R8)
- **Load -** moving data from memory location to a register.
- **Store -** moving data from a register to memory location.
- **Transfer -** moving data from register to register.
- **Branch -** the next instruction is somewhere else in memory.
  - **Conditional branch -** the next instruction might be somewhere else, or it is the next instruction.

Lecture 18

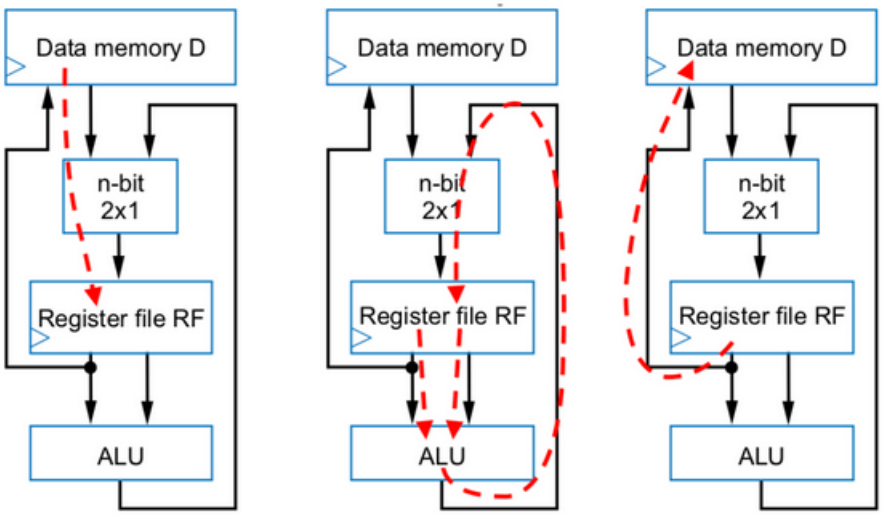
- **Machine Code** – bit encoding of instruction operations and operands
- **Assembly Language** – mnemonic-based representation of the instruction → different versions exist for different ISAs
- **Compiler** – transforms high level software language into assembly language
- **Assembler** – transforms assembly language into machine code

Key Takeaways

- Moving towards creating a CPU layout
- Each instruction that we as programmers call triggers a set of signals that completes our action in the datapath.
- Assembly language is an abstract level above machine language because people can't/don't want to write in 0s and 1s.
- How to read and write assembly commands in both assembly language and machine code.
- Need to pick an arbitrary memory address to start the program at
- Since opcodes end up as bits, we can define any instruction we want → Lab 6
- Datapath for CPU
- How to move the PC

Questions

Question 1: For the given basic datapath, show the path for a i.) load operation, ii.) ALU operation, iii.) store operation.



Question 2: Convert this question into Assembly

```
#define LEN(8)
uint16_t nums[LEN];
  for (int i = 0; i < LEN; ++i) {
    nums[i] = i;
  }
```

Let the start of array nums be located at address $8000. Write the assembly to "label" an address in memory to be nums.

```
        .ORG    $8000
nums    .DW     $0
```

Now write the code to initialize the iterator i. This code should start at address $0100. We can either keep the value of i stored in a register, or in memory. Given the time it takes to execute certain instructions, one of these locations for i is clearly better than the other. I'll let you figure that out.

```
.ORG    $100            ; start code at $0100
MV      r1, r0          ; load i=0 into r1
```

Finally, write the rest of the code to loop and set the elements of the array.

```
        ADDI    r2, r0, $8      ; upper bound of loop, LEN=8
loop    SUB     r0, r2, r1      ; check 8-i, result doesn't matter
        BRZ     done            ; if (8-i == 0), then we're done
        SLLI    r3, r1, $1      ; multiply i by 2 for word alignment
        SW      r3, r1, nums    ; nums[&nums + i*2] = i
        ADDI    r1, r1, $1      ; i++
        BRA     loop            ; go back to loop
done    STOP
```

Question 3: Assuming each load/ALU/store operation takes a single cycle, find the total number of cycles for the below operations.

a) Copy data from a memory location into another memory location.

Data must first be loaded into register file then stored into destination memory -> 2 cycles

b) Copy two register locations into two memory locations

Only one register file to memory location copy is permitted during a single cycle -> 2 cycles

c) Add data from a register file location and a memory location, storing the result in the same memory location

Data must first be loaded into a register file, then addition must be performed, then the sum must be stored into memory location -> 3 cycles

d) Copy data from a register file location into a memory location

1 cycle

e) Subtract data from two memory locations and store the result in another memory location
2 cycles are required to load the two operands, 1 cycle to perform subtraction, 1 cycle to store difference (4 cycles)

Question 4: Evaluate the following portion of assembly code.

a) To help with understanding, write a small comment beside each line saying what it does.

```
        .ORG    $2000
5 references
START   ADDI    r7, r0, $5000
        MV      r2, r0

2 references
A       LW      r1, r7, $0
        SLTI    r0, r1, $0020
        BRNZ    B
        BRA     C

1 reference
B       ADD     r2, r2, r1

2 references
C       ADDI    r7, r7, $2
        SLTI    r0, r7, $5010
        BRNZ    A

1 reference
D       SW      r7, r2, $0
        STOP
```

Specify where your program starts

load the address of the first value ($5000) into r7 (r7 is address counter)

load zero into r2 (initialize sum = 0)

load value stored at the address from r7 to r1

check if value is less than 0x20

if less than 0x20, go to B.
if not, go to C

add r1 to r2, store back in r2 (sum = sum + r1)
r2 acts as rolling sum

increment address by 2 to get to next value in memory

check if we passed stop address ($5010)

if not, loop back up to A
if we did pass it, fall down to D

store the calculated sum into memory at the address after the data ($5012)

stop (self-explanatory)

b) What is the general task that this code is doing?

The code is reading 9 consecutive values from memory and summing any values that are less than 0x20. Then, it stores the sum back to memory.

c) How many bytes of memory does the code for this program take up? Show your work.

Take each instruction, determine whether it is a one- or two-word instruction, find number of bytes (1 word = 2 bytes), sum up all values.

ADDI → 2 words (4 bytes)
MV → 1 word (2 bytes)
LW → 2 words (4 bytes)
SLTI → 2 words (4 bytes)
BRNZ → 2 words (4 bytes)
BRA → 2 words (4 bytes)
ADD → 1 word (2 bytes)
ADDI → 2 words (4 bytes)
SLTI → 2 words (4 bytes)
BRNZ → 2 words (4 bytes)
SW → 2 words (4 bytes)
STOP → 1 word (2 bytes)

⟹ **42 bytes total**